Развој софтвера 2

Увод

Стефан Капунац

Математички факултет, Универзитет у Београду

школска 2026/27. година

Организација курса

- Пројекат - 100 поена

Развој софтвера 1 и 2

Развој софтвера

- добро организован програм
- пројектовање класа и модула
- обрасци, тестови, рефакторисање

Развој софтвера 2

- систем од више компоненти
- тимови, кварови, мрежа, продукција
- архитектурне одлуке и trade-off-ови

Предавања и вежбе

Вежбе

- конкретна .NET апликација
- Catalog, Basket, Discount, Ordering
- Docker, gRPC, RabbitMQ, gateway, JWT

Предавања

- зашто баш таква решења?
- које су алтернативе?
- шта се дешава у продукцији?

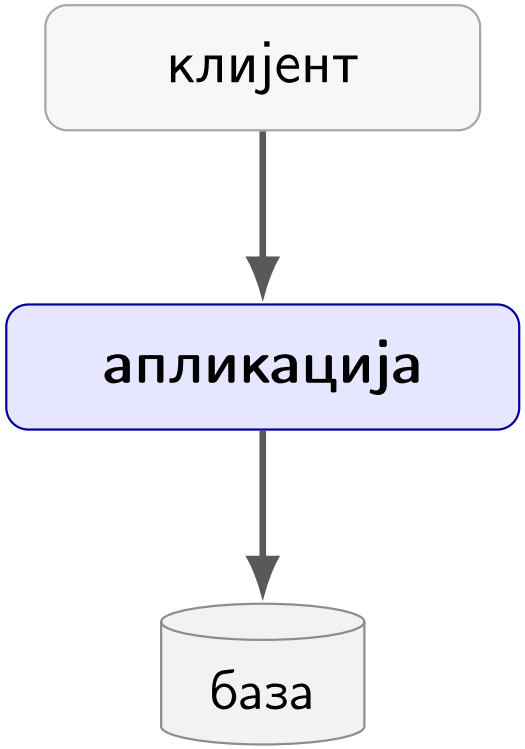
Једноставан почетак

Почетни систем: мала продавница

- 1000 корисника
- 3 програмера
- једна држава
- 20 000 производа

Питање

Да ли је ово лоша архитектура?



Није сваки монолит проблем

- Једноставна архитектура може бити најбољи избор
- Један deployment, једна база, мање инфраструктуре
- Лакше је разумети, тестирати и дебаговати систем

Важна порука

Архитектура није добра или лоша сама по себи. Добра је ако одговара контексту.

Исти функционални захтеви, други контекст

Мала продавница

- стотине корисника
- један тим
- неколико измена месечно

Велика платформа

- милиони корисника
- много тимова
- много измена дневно

Функционални захтеви нису довољни да одреде архитектуру.

Шта још морамо да знамо?

- перформансе
- скалабилност
- доступност
- безбедност
- одрживост
- тестирање
- испорука нових верзија
- посматрање система у раду

Систем расте

Први проблем: код постаје велики

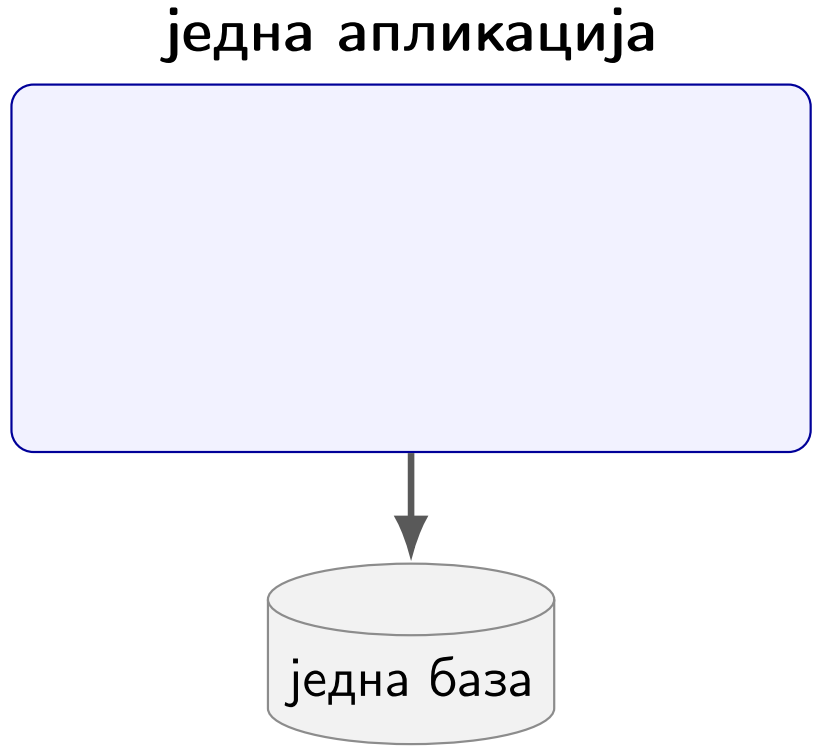
- Каталог се мења из једног разлога
- Плаћање се мења из другог разлога
- Корпа се мења из трећег разлога
- Један велики пројекат постаје тежак за разумевање

Питање

Како поделити систем, а да га не учинимо непотребно сложеним?

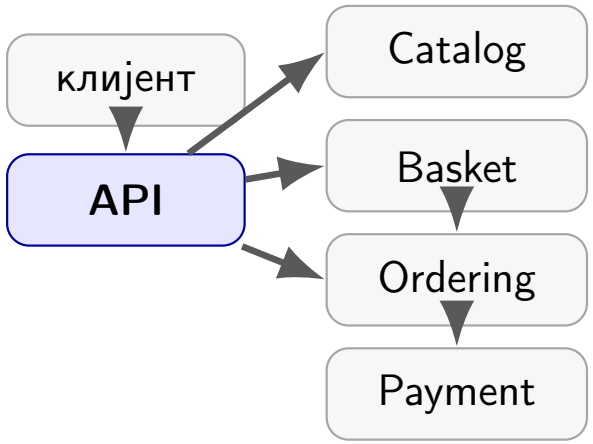
Модуларни монолит

- једна апликација
- један deployment
- модули имају јасне границе
- позиви су локални, без мреже



Микросервиси

- више независних сервиса
- сваки сервис се посебно deploy-yje
- често сваки сервис има своје податке
- комуникација иде преко мреже



Модуларни монолит vs микросервиси

Модуларни монолит

- једна апликација
- један deployment
- локални позиви
- границе су у коду

Микросервиси

- више апликација/процеса
- независни deployment-и
- мрежни позиви
- границе су API-ји и одговорности

Кључна разлика

Код модуларног монолита систем се deploy-ује као целина. Код микросервиса сваки сервис живи независније, али цена је дистрибуирана сложеност.

Где су границе?

Catalog
Product
име
опис
слике

Inventory
Product
SKU
количина
магацин

Pricing
Product
цена
попуст
валута

Да ли је ово један исти модел?

DDD као начин размишљања о границама

- Различити делови система често имају различит језик
- Иста реч може имати различито значење у различитом контексту
- Границе домена могу постати границе модула или сервиса

Зашто је Netflix мењао архитектуру?

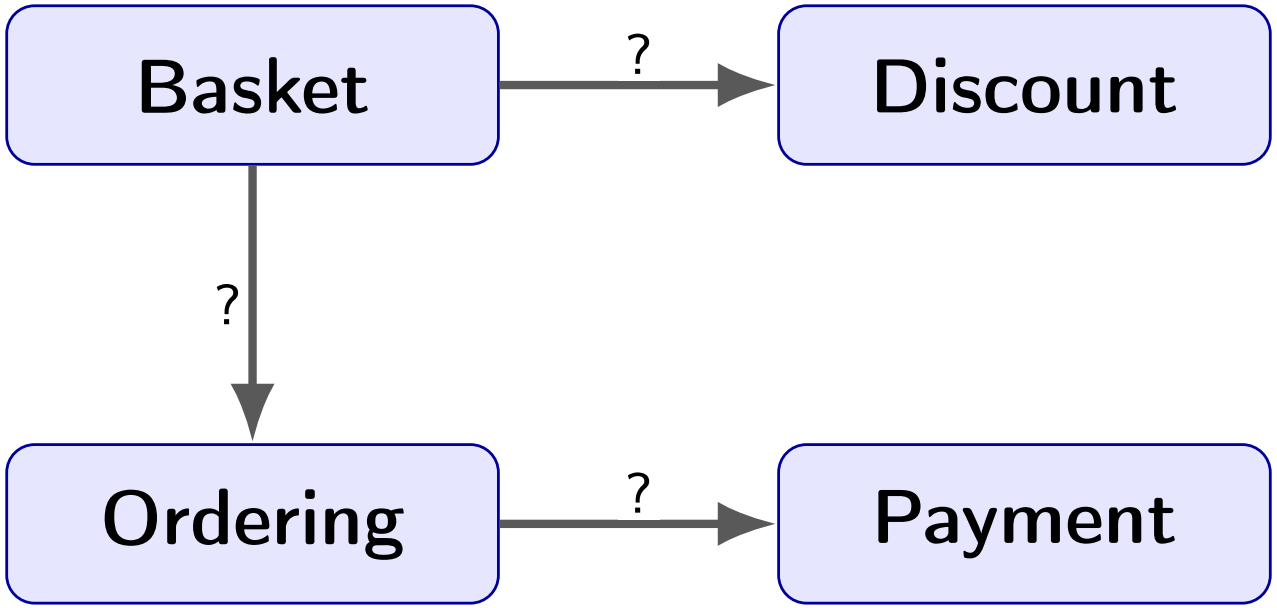
- 2008: велики проблем са базом и слањем DVD-ова
- жеља да се избегну вертикално скалирани single points of failure
- прелаз на cloud, хоризонтално скалирање и дистрибуиране системе
- касније: стотине микросервиса и независнији тимови

Али

Netflix-ов проблем није нужно наш проблем.

Компоненте морају да комуницирају

Када раздвојимо компоненте



Некада је био позив методе. Сада је мрежа.

Типови комуникације између компоненти

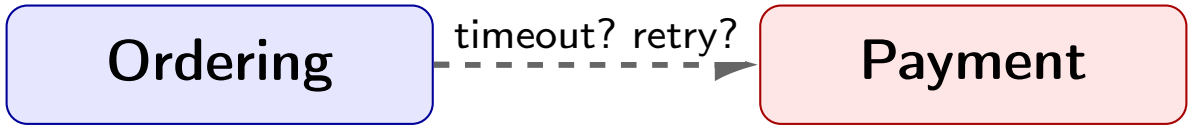
- **REST** – request/response API заснован на ресурсима
- **GraphQL** – клијент бира облик и поља одговора
- **gRPC** – типизирана, ефикасна service-to-service комуникација
- **Messaging** – асинхрона комуникација преко порука и догађаја

Идеја

Не постоји један најбољи избор. Зависи од клијената, домена, перформанси, поузданости и начина рада тимова.

Сада смо у продукцији

Payment не одговара



- Шта ако одговор касни 30 секунди?
- Шта ако је уплата успела, али одговор није стигао?
- Шта ако retry наплати кориснику два пута?

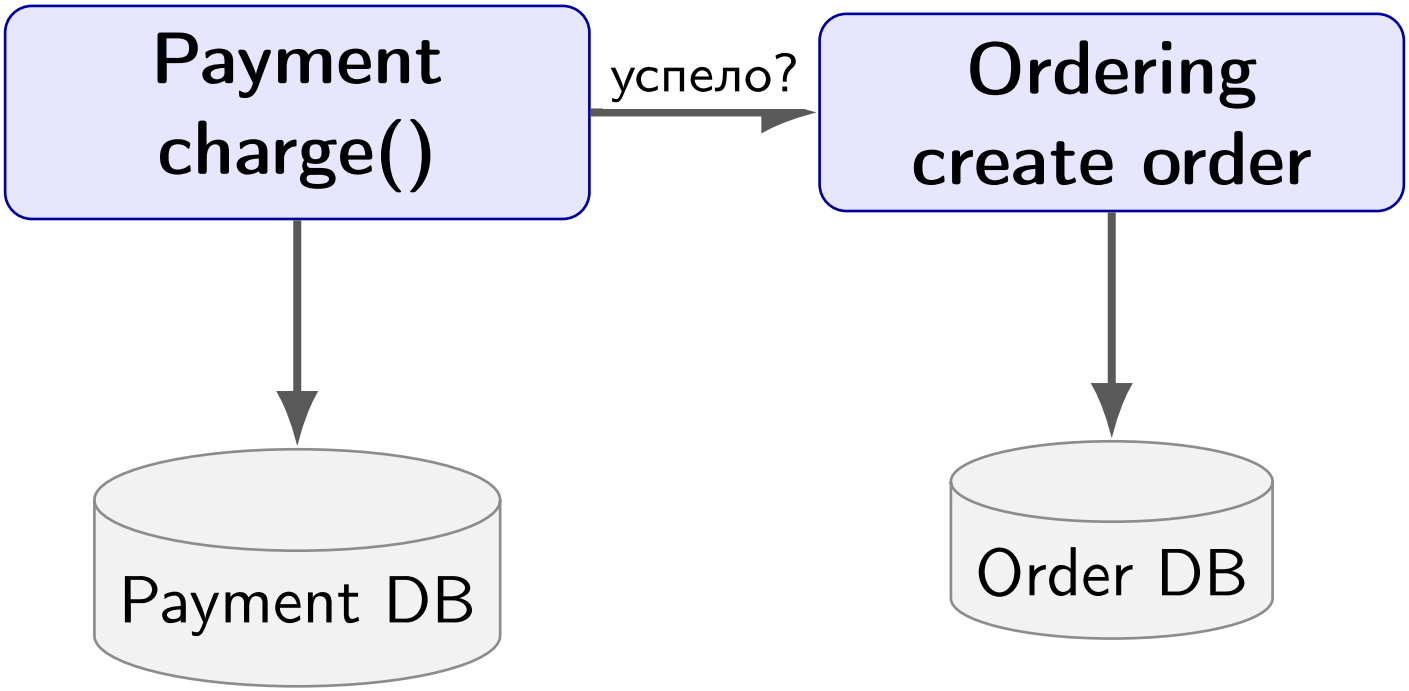
Дистрибуирани системи мењају правила

- мрежа може да закаже
- сервис може да буде спор, а не мртав
- порука може да стигне два пута
- поруке могу да стигну другим редом
- није увек јасно шта се тачно десило

Последица

Морамо да пројектујемо систем као да ће кварови бити нормалан случај, не изузетак.

Подаци и конзистентност



Питање

Шта ако је корисник платио, али поруџбина није креирана?

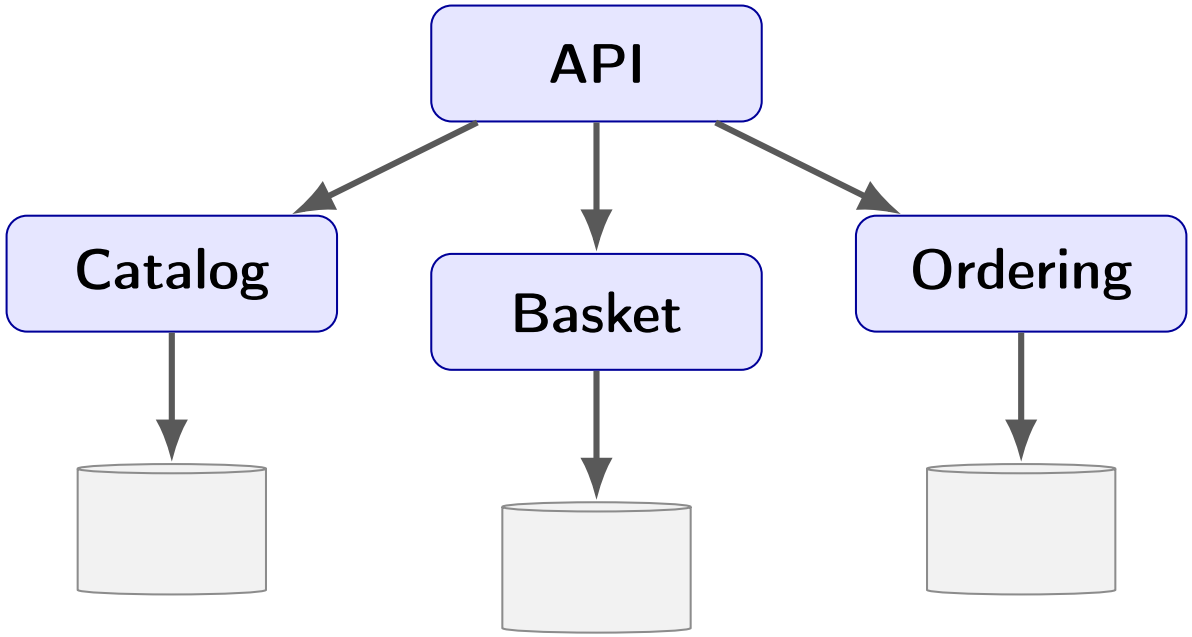
Поузданост: систем који очекује квар

- timeout
- retry са backoff-ом
- idempotency key
- circuit breaker
- rate limiting
- bulkhead
- graceful degradation
- dead-letter queue

Циљ

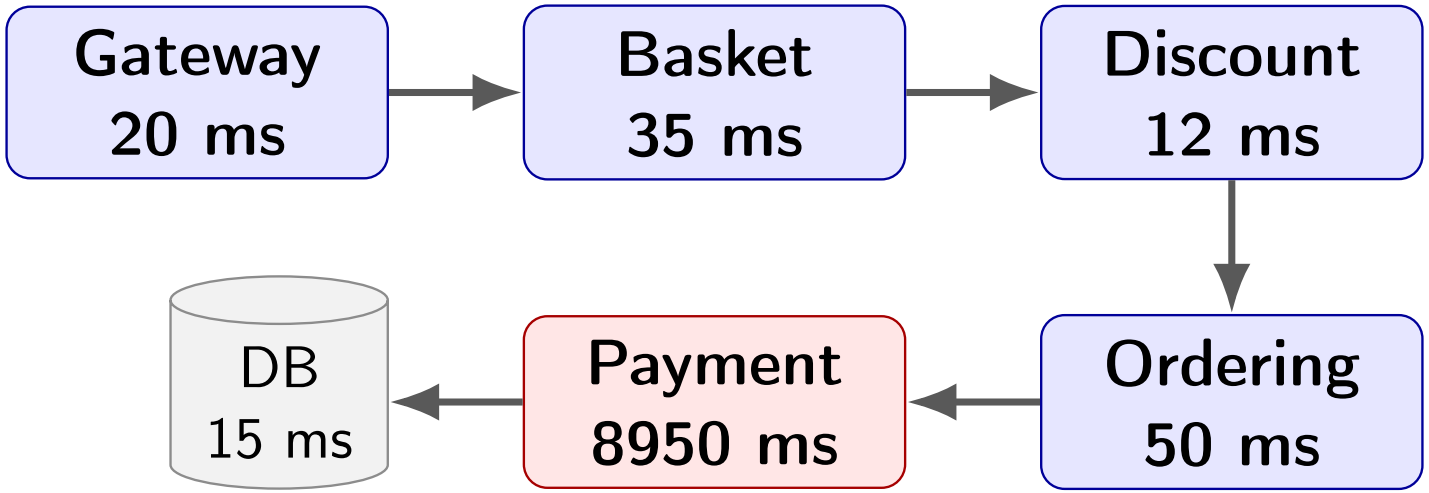
Не правимо систем који никад не пада. Правимо систем који контролисано преживљава кварове.

Како ово тестирати?



- Ако све mock-ујемо, да ли смо тестирали интеграцију?
- Ако сваки тест диже цео систем, да ли ће икада бити брз?
- Како проверити да се два сервиса слажу око API уговора?

Корисник каже: checkout траје 12 секунди

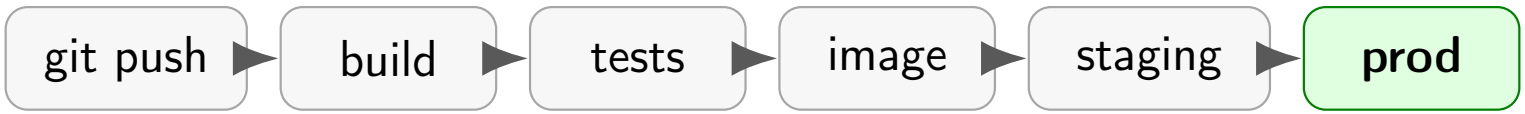


Где је проблем?

Observability: како налазимо проблем?

- **Logs** – шта се десило у појединачним компонентама?
- **Metrics** – да ли је ово системски проблем или изолован случај?
- **Tracing** – кроз које сервисе је прошао конкретан захтев?
- **Alerting** – како сазнамо за проблем пре корисника?

Од commit-а до продукције



Окружења

local, dev, staging, production

CI/CD

build, test, artifact, deploy, rollback

Deployment без прекида рада

Rolling

v1 v1 v1
v2 v1 v1
v2 v2 v1
v2 v2 v2

Blue/green

једно окружење ради
друго чека спремно

Canary

95% корисника: v1
5% корисника: v2

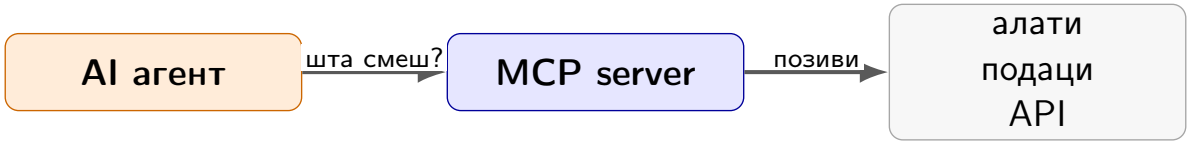
Пет година касније

- стари клијенти и нови клијенти
- legacy API-ји
- миграције базе
- backward compatibility
- технички дуг

Проблем

Систем не треба само направити. Треба га годинама мењати без заустављања посла.

Нови клијент: AI агент



- Како агент открива могућности система?
- Како контролишемо дозволе?
- Како пратимо шта је агент урадио?

Мапа курса

Мапа курса: првих 6 недеља

1. Quality attributes и архитектурни trade-off-ови
2. Архитектура и модуларност
3. Монолит, модуларни монолит и микросервиси
4. REST, GraphQL, gRPC и messaging
5. Основе дистрибуираних система
6. Подаци и конзистентност

Мапа курса: других 6 недеља

7. Resilience и скалирање
8. Тестирање дистрибуираних система
9. Observability: logs, metrics, tracing, alerting
10. Окружења и CI/CD
11. Deployment и продукција
12. Еволуција система, AI агенти и MCP

Шта треба да остане после данашњег часа?

1. Технологија је одговор на проблем, не циљ сама по себи
2. Модуларни монолит и микросервиси решавају различите проблеме
3. Чим систем постане дистрибуиран, добијамо нове класе кварова
4. Продукција је део архитектуре, не накнадна активност

Референце за примере

- Netflix Tech Blog / About Netflix: *Completing the Netflix Cloud Migration*, 2016
- Martin Fowler: *Microservice Trade-Offs*, 2015
- Martin Fowler: *Monolith First*, 2015
- OpenTelemetry: *Observability Primer*